UNIQ ID: E0124092

```
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------
# Multi-Armed Bandit Environment
# ------------------------------

np.random.seed(42)

n_arms = 5
n_rounds = 1000

# True probability of getting reward = 1 for each arm
true_probabilities = [0.1, 0.3, 0.5, 0.7, 0.9]

# ------------------------------
# UCB Algorithm
# ------------------------------

def ucb(n_arms, n_rounds):
    counts = np.zeros(n_arms)
    rewards = np.zeros(n_arms)

    total_rewards = []
    selected_arms = []

    for t in range(n_rounds):

        # Select each arm once initially
        if t < n_arms:
            arm = t

        else:
            ucb_values = (
                rewards / counts
                + np.sqrt(2 * np.log(t + 1) / counts)
            )
            arm = np.argmax(ucb_values)

        # Generate reward
        reward = np.random.binomial(1, true_probabilities[arm])

        # Update
        counts[arm] += 1
        rewards[arm] += reward

        total_rewards.append(reward)
        selected_arms.append(arm)

    return total_rewards, selected_arms


# ------------------------------
# Thompson Sampling Algorithm
# ------------------------------

def thompson_sampling(n_arms, n_rounds):

    # Initially Beta(1,1) for every arm
    alpha = np.ones(n_arms)
    beta = np.ones(n_arms)

    total_rewards = []
    selected_arms = []

    for t in range(n_rounds):

        # Sample from Beta distribution
        samples = np.random.beta(alpha, beta)

        # Select arm with highest sample
```

```
        arm = np.argmax(samples)

        # Generate reward
        reward = np.random.binomial(1, true_probabilities[arm])

        # Update Beta distribution
        if reward == 1:
            alpha[arm] += 1
        else:
            beta[arm] += 1

        total_rewards.append(reward)
        selected_arms.append(arm)

    return total_rewards, selected_arms


# ------------------------------
# Run Algorithms
# ------------------------------

ucb_rewards, ucb_arms = ucb(n_arms, n_rounds)

ts_rewards, ts_arms = thompson_sampling(n_arms, n_rounds)


# ------------------------------
# Calculate Cumulative Rewards
# ------------------------------

ucb_cumulative = np.cumsum(ucb_rewards)
ts_cumulative = np.cumsum(ts_rewards)


# ------------------------------
# Display Results
# ------------------------------

print("True probabilities of arms:")
for i, p in enumerate(true_probabilities):
    print(f"Arm {i+1}: {p}")

print("\nUCB Results")
print("-----------")
print("Total Reward:", sum(ucb_rewards))
print("Best Arm Selected:", np.argmax(np.bincount(ucb_arms)) + 1)

print("\nThompson Sampling Results")
print("-------------------------")
print("Total Reward:", sum(ts_rewards))
print("Best Arm Selected:", np.argmax(np.bincount(ts_arms)) + 1)


# ------------------------------
# Plot Cumulative Rewards
# ------------------------------

plt.figure(figsize=(10, 5))

plt.plot(
    ucb_cumulative,
    label="UCB"
)

plt.plot(
    ts_cumulative,
    label="Thompson Sampling"
)

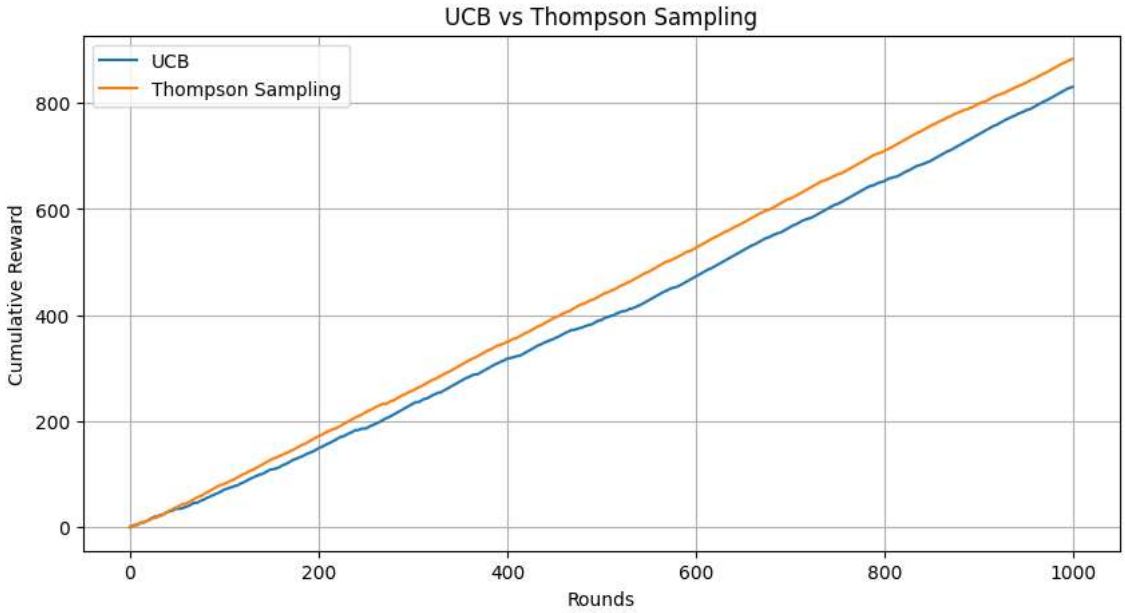
plt.xlabel("Rounds")
plt.ylabel("Cumulative Reward")
plt.title("UCB vs Thompson Sampling")
plt.legend()
plt.grid()

plt.show()
```

```
True probabilities of arms:
Arm 1: 0.1
Arm 2: 0.3
Arm 3: 0.5
Arm 4: 0.7
Arm 5: 0.9

UCB Results
-----------
Total Reward: 830
Best Arm Selected: 5

Thompson Sampling Results
-------------------------
Total Reward: 883
Best Arm Selected: 5
```



Start coding or generate with AI.